

Programación Orientada a Objetos (POO) en PHP

1. Introducción a la POO

¿Qué es la POO?

La Programación Orientada a Objetos es un paradigma de programación que organiza el código en **objetos** que contienen datos (propiedades) y comportamientos (métodos).

Conceptos Fundamentales

- **Clase:** Plantilla o molde para crear objetos
- **Objeto:** Instancia de una clase
- **Propiedad:** Variable dentro de una clase
- **Método:** Función dentro de una clase
- **Instancia:** Objeto creado a partir de una clase

Ventajas de la POO

- Reutilización de código
- Mejor organización
- Facilita el mantenimiento
- Abstracción de la complejidad
- Modularidad

2. Clases y Objetos

Crear una Clase

```
<?php
class Persona {
    // Propiedades
    public $nombre;
    public $edad;

    // Método
    public function saludar() {
        return "Hola, soy " . $this->nombre;
    }
}
```

Crear Objetos (Instancias)

```
<?php
// Crear objeto
$persona1 = new Persona();

// Asignar valores a las propiedades
$persona1->nombre = "Juan";
```

```
$persona1->edad = 25;

// Llamar a un método
echo $persona1->saludar(); // "Hola, soy Juan"

// Crear otro objeto
$persona2 = new Persona();
$persona2->nombre = "María";
$persona2->edad = 30;

echo $persona2->saludar(); // "Hola, soy María"
?>
```

La Palabra Clave `$this`

`$this` hace referencia al objeto actual dentro de la clase.

```
<?php
class Calculadora {
    public $numero1;
    public $numero2;

    public function sumar() {
        return $this->numero1 + $this->numero2;
    }

    public function restar() {
        return $this->numero1 - $this->numero2;
    }
}

$calc = new Calculadora();
$calc->numero1 = 10;
$calc->numero2 = 5;

echo $calc->sumar(); // 15
echo $calc->restar(); // 5
?>
```

3. Constructor y Destructor

Constructor `__construct()`

El constructor es un método especial que se ejecuta automáticamente al crear un objeto.

```
<?php
class Persona {
    public $nombre;
    public $edad;
```

```

// Constructor
public function __construct($nombre, $edad) {
    $this->nombre = $nombre;
    $this->edad = $edad;
    echo "Objeto creado: " . $this->nombre . "<br>";
}

public function presentarse() {
    return "Soy {$this->nombre} y tengo {$this->edad} años";
}
}

// Crear objeto pasando parámetros al constructor
$persona = new Persona("Juan", 25);
echo $persona->presentarse();
?>

```

Constructor con Valores por Defecto

```

<?php
class Producto {
    public $nombre;
    public $precio;
    public $stock;

    public function __construct($nombre, $precio = 0, $stock = 0) {
        $this->nombre = $nombre;
        $this->precio = $precio;
        $this->stock = $stock;
    }
}

$producto1 = new Producto("Laptop", 800, 10);
$producto2 = new Producto("Mouse"); // precio y stock serán 0
?>

```

Destructor `__destruct()`

El destructor se ejecuta cuando el objeto es destruido o el script termina.

```

<?php
class Conexion {
    private $nombre;

    public function __construct($nombre) {
        $this->nombre = $nombre;
        echo "Conexión abierta: {$this->nombre}<br>";
    }

    public function __destruct() {

```

```

        echo "Conexión cerrada: {$this->nombre}<br>";
    }
}

$conn = new Conexion("Base de datos");
// Al finalizar el script, se llama automáticamente al destructor
?>

```

4. Modificadores de Acceso

Tipos de Modificadores

- **public:** Accesible desde cualquier lugar
- **private:** Solo accesible desde dentro de la clase
- **protected:** Accesible desde la clase y sus hijos

Ejemplo de Modificadores

```

<?php
class CuentaBancaria {
    public $titular;           // Accesible desde cualquier lugar
    private $saldo;           // Solo accesible dentro de la clase
    protected $numeroCuenta;  // Accesible en la clase y clases hijas

    public function __construct($titular, $saldoInicial) {
        $this->titular = $titular;
        $this->saldo = $saldoInicial;
        $this->numeroCuenta = rand(1000, 9999);
    }

    // Método público para acceder al saldo privado
    public function getSaldo() {
        return $this->saldo;
    }

    // Método público para modificar el saldo
    public function depositar($cantidad) {
        if ($cantidad > 0) {
            $this->saldo += $cantidad;
            return true;
        }
        return false;
    }

    public function retirar($cantidad) {
        if ($cantidad > 0 && $cantidad <= $this->saldo) {
            $this->saldo -= $cantidad;
            return true;
        }
        return false;
    }
}

```

```

}

$cuenta = new CuentaBancaria("Juan", 1000);

echo $cuenta->titular; // ✓ OK - es public
// echo $cuenta->saldo; // ✗ Error - es private

echo $cuenta->getSaldo(); // ✓ OK - método público
$cuenta->depositar(500);
echo $cuenta->getSaldo(); // 1500
?>

```

Getters y Setters

```

<?php
class Usuario {
    private $nombre;
    private $email;
    private $edad;

    // Getter para nombre
    public function getNombre() {
        return $this->nombre;
    }

    // Setter para nombre
    public function setNombre($nombre) {
        // Validación
        if (strlen($nombre) >= 3) {
            $this->nombre = $nombre;
        } else {
            throw new Exception("El nombre debe tener al menos 3 caracteres");
        }
    }

    // Getter para email
    public function getEmail() {
        return $this->email;
    }

    // Setter para email con validación
    public function setEmail($email) {
        if (filter_var($email, FILTER_VALIDATE_EMAIL)) {
            $this->email = $email;
        } else {
            throw new Exception("Email inválido");
        }
    }

    // Getter para edad
    public function getEdad() {

```

```

        return $this->edad;
    }

    // Setter para edad con validación
    public function setEdad($edad) {
        if ($edad >= 0 && $edad <= 150) {
            $this->edad = $edad;
        } else {
            throw new Exception("Edad inválida");
        }
    }
}

$usuario = new Usuario();
$usuario->setNombre("Juan");
$usuario->setEmail("juan@example.com");
$usuario->setEdad(25);

echo $usuario->getNombre(); // Juan
?>

```

5. Herencia

Concepto de Herencia

La herencia permite crear una clase basada en otra clase existente, heredando sus propiedades y métodos.

Sintaxis Básica

```

<?php
// Clase padre (superclase)
class Animal {
    public $nombre;
    public $edad;

    public function __construct($nombre, $edad) {
        $this->nombre = $nombre;
        $this->edad = $edad;
    }

    public function comer() {
        return "{$this->nombre} está comiendo";
    }

    public function dormir() {
        return "{$this->nombre} está durmiendo";
    }
}

// Clase hija (subclase) que hereda de Animal

```

```

class Perro extends Animal {
    public $raza;

    public function __construct($nombre, $edad, $raza) {
        parent::__construct($nombre, $edad); // Llamar al constructor padre
        $this->raza = $raza;
    }

    // Método específico de Perro
    public function ladrar() {
        return "{$this->nombre} está ladrando: ¡Guau!";
    }
}

// Usar la clase hija
$perro = new Perro("Rex", 3, "Pastor Alemán");
echo $perro->comer(); // Método heredado
echo $perro->ladrar(); // Método propio
?>

```

La Palabra Clave `parent`

```

<?php
class Vehiculo {
    protected $marca;
    protected $modelo;

    public function __construct($marca, $modelo) {
        $this->marca = $marca;
        $this->modelo = $modelo;
    }

    public function obtenerInfo() {
        return "Vehículo: {$this->marca} {$this->modelo}";
    }
}

class Coche extends Vehiculo {
    private $numeroPuertas;

    public function __construct($marca, $modelo, $numeroPuertas) {
        parent::__construct($marca, $modelo); // Llamar constructor padre
        $this->numeroPuertas = $numeroPuertas;
    }

    // Sobrescribir método del padre
    public function obtenerInfo() {
        $info = parent::obtenerInfo(); // Obtener info del padre
        return $info . " - Puertas: {$this->numeroPuertas}";
    }
}

```

```
$coche = new Coche("Toyota", "Corolla", 4);
echo $coche->obtenerInfo();
// "Vehículo: Toyota Corolla – Puertas: 4"
?>
```

Ejemplo Completo de Herencia

```
<?php
// Clase base
class Empleado {
    protected $nombre;
    protected $salarioBase;

    public function __construct($nombre, $salarioBase) {
        $this->nombre = $nombre;
        $this->salarioBase = $salarioBase;
    }

    public function calcularSalario() {
        return $this->salarioBase;
    }

    public function obtenerInfo() {
        return "Empleado: {$this->nombre} – Salario: €{$this->calcularSalario()}";
    }
}

// Clase derivada 1
class Gerente extends Empleado {
    private $bono;

    public function __construct($nombre, $salarioBase, $bono) {
        parent::__construct($nombre, $salarioBase);
        $this->bono = $bono;
    }

    public function calcularSalario() {
        return $this->salarioBase + $this->bono;
    }
}

// Clase derivada 2
class Desarrollador extends Empleado {
    private $horasExtra;
    private $pagoPorHora;

    public function __construct($nombre, $salarioBase, $horasExtra, $pagoPorHora) {
        parent::__construct($nombre, $salarioBase);
        $this->horasExtra = $horasExtra;
        $this->pagoPorHora = $pagoPorHora;
    }
}
```



```

    }

    public function calcularSalario() {
        return $this->salarioBase + ($this->horasExtra * $this->pagoPorHora);
    }
}

// Usar las clases
$empleado = new Empleado("Carlos", 2000);
$gerente = new Gerente("Ana", 3000, 1000);
$dev = new Desarrollador("Luis", 2500, 20, 30);

echo $empleado->obtenerInfo() . "<br>"; // Salario: €2000
echo $gerente->obtenerInfo() . "<br>";   // Salario: €4000
echo $dev->obtenerInfo() . "<br>";       // Salario: €3100
?>

```

6. Encapsulación

¿Qué es la Encapsulación?

La encapsulación es ocultar los detalles internos de una clase y proporcionar una interfaz pública para interactuar con ella.

Ejemplo Práctico

```

<?php
class CuentaBancaria {
    private $saldo;
    private $titular;
    private $transacciones = [];

    public function __construct($titular, $saldoInicial = 0) {
        $this->titular = $titular;
        $this->saldo = $saldoInicial;
        $this->registrarTransaccion("Apertura de cuenta", $saldoInicial);
    }

    public function depositar($cantidad) {
        if ($cantidad <= 0) {
            throw new Exception("La cantidad debe ser positiva");
        }

        $this->saldo += $cantidad;
        $this->registrarTransaccion("Depósito", $cantidad);
        return true;
    }

    public function retirar($cantidad) {
        if ($cantidad <= 0) {

```

```

        throw new Exception("La cantidad debe ser positiva");
    }

    if ($cantidad > $this->saldo) {
        throw new Exception("Saldo insuficiente");
    }

    $this->saldo -= $cantidad;
    $this->registrarTransaccion("Retiro", -$cantidad);
    return true;
}

public function getSaldo() {
    return $this->saldo;
}

public function getTitular() {
    return $this->titular;
}

public function getTransacciones() {
    return $this->transacciones;
}

// Método privado, no accesible desde fuera
private function registrarTransaccion($tipo, $cantidad) {
    $this->transacciones[] = [
        'fecha' => date('Y-m-d H:i:s'),
        'tipo' => $tipo,
        'cantidad' => $cantidad,
        'saldo_resultante' => $this->saldo
    ];
}
}

// Uso
$cuenta = new CuentaBancaria("Juan Pérez", 1000);
$cuenta->depositar(500);
$cuenta->retirar(200);

echo "Saldo: €" . $cuenta->getSaldo() . "<br>";

foreach ($cuenta->getTransacciones() as $trans) {
    echo "{$trans['fecha']} - {$trans['tipo']}: €{$trans['cantidad']}<br>";
}
?>

```

7. Métodos y Propiedades Estáticas

Propiedades Estáticas

Las propiedades estáticas pertenecen a la clase, no a las instancias.

```
<?php
class Contador {
    public static $cuenta = 0;

    public function __construct() {
        self::$cuenta++; // Incrementar el contador
    }

    public static function obtenerCuenta() {
        return self::$cuenta;
    }
}

// Acceder sin crear instancia
echo Contador::$cuenta . "<br>"; // 0

// Crear instancias
$obj1 = new Contador();
$obj2 = new Contador();
$obj3 = new Contador();

echo Contador::obtenerCuenta(); // 3
?>
```

Métodos Estáticos

```
<?php
class Utilidades {
    public static function saludar($nombre) {
        return "Hola, {$nombre}";
    }

    public static function sumar($a, $b) {
        return $a + $b;
    }

    public static function esPar($numero) {
        return $numero % 2 === 0;
    }
}

// Llamar a métodos estáticos sin crear instancia
echo Utilidades::saludar("Juan") . "<br>";
echo Utilidades::sumar(5, 3) . "<br>";
echo Utilidades::esPar(10) ? "Es par" : "Es impar";
?>
```

Ejemplo Práctico: Clase de Configuración

```
<?php
class Config {
    private static $configuracion = [];

    public static function set($clave, $valor) {
        self::$configuracion[$clave] = $valor;
    }

    public static function get($clave) {
        return self::$configuracion[$clave] ?? null;
    }

    public static function existe($clave) {
        return isset(self::$configuracion[$clave]);
    }
}

// Usar la configuración
Config::set('app_name', 'Mi Aplicación');
Config::set('version', '1.0');
Config::set('debug', true);

echo Config::get('app_name'); // Mi Aplicación
echo Config::existe('version') ? 'Existe' : 'No existe'; // Existe
?>
```

self vs \$this

```
<?php
class Ejemplo {
    private $propiedadInstancia = "Instancia";
    private static $propiedadEstatica = "Estática";

    public function metodoInstancia() {
        // Acceder a propiedad de instancia
        echo $this->propiedadInstancia . "<br>";

        // Acceder a propiedad estática
        echo self::$propiedadEstatica . "<br>";
    }

    public static function metodoEstatico() {
        // No se puede usar $this en métodos estáticos
        // echo $this->propiedadInstancia; // x Error

        // Solo se puede acceder a miembros estáticos
        echo self::$propiedadEstatica . "<br>";
    }
}
```

```
}  
?>
```

8. Constantes de Clase

Definir Constantes

```
<?php  
class Matematicas {  
    const PI = 3.14159;  
    const E = 2.71828;  
  
    public static function areaCirculo($radio) {  
        return self::PI * $radio * $radio;  
    }  
}  
  
// Acceder a constantes  
echo Matematicas::PI . "<br>";  
echo Matematicas::areaCirculo(5);  
?>
```

Ejemplo con Configuración

```
<?php  
class BaseDatos {  
    const HOST = "localhost";  
    const USUARIO = "root";  
    const PASSWORD = "";  
    const NOMBRE_BD = "mi_base_datos";  
  
    public static function conectar() {  
        return new mysqli(  
            self::HOST,  
            self::USUARIO,  
            self::PASSWORD,  
            self::NOMBRE_BD  
        );  
    }  
}  
  
$conn = BaseDatos::conectar();  
?>
```

9. Clases Abstractas

¿Qué es una Clase Abstracta?

Una clase abstracta no se puede instanciar directamente. Sirve como plantilla para otras clases.

```
<?php
abstract class FiguraGeometrica {
    protected $nombre;

    public function __construct($nombre) {
        $this->nombre = $nombre;
    }

    // Método abstracto (debe ser implementado por clases hijas)
    abstract public function calcularArea();

    // Método concreto (puede ser usado por clases hijas)
    public function obtenerNombre() {
        return $this->nombre;
    }
}

class Rectangulo extends FiguraGeometrica {
    private $base;
    private $altura;

    public function __construct($base, $altura) {
        parent::__construct("Rectángulo");
        $this->base = $base;
        $this->altura = $altura;
    }

    // Implementar método abstracto
    public function calcularArea() {
        return $this->base * $this->altura;
    }
}

class Circulo extends FiguraGeometrica {
    private $radio;

    public function __construct($radio) {
        parent::__construct("Círculo");
        $this->radio = $radio;
    }

    public function calcularArea() {
        return pi() * $radio * $this->radio;
    }
}

// $figura = new FiguraGeometrica("Test"); // x Error - no se puede instanciar

$rectangulo = new Rectangulo(5, 10);
```

```
echo $rectangulo->obtenerNombre() . ": " . $rectangulo->calcularArea() . "<br>";

$circulo = new Circulo(7);
echo $circulo->obtenerNombre() . ": " . $circulo->calcularArea();
?>
```

10. Interfaces

¿Qué es una Interface?

Una interface define un contrato que las clases deben cumplir. Solo contiene declaraciones de métodos, sin implementación.

```
<?php
interface Reproducible {
    public function reproducir();
    public function pausar();
    public function detener();
}

interface Descargable {
    public function descargar();
}

class Video implements Reproducible, Descargable {
    private $titulo;
    private $duracion;

    public function __construct($titulo, $duracion) {
        $this->titulo = $titulo;
        $this->duracion = $duracion;
    }

    public function reproducir() {
        return "Reproduciendo video: {$this->titulo}";
    }

    public function pausar() {
        return "Video pausado";
    }

    public function detener() {
        return "Video detenido";
    }

    public function descargar() {
        return "Descargando video: {$this->titulo}";
    }
}

$video = new Video("Tutorial PHP", "15:30");
```

```
echo $video->reproducir() . "<br>";
echo $video->descargar();
?>
```

Diferencia entre Clase Abstracta e Interface

Clase Abstracta	Interface
Puede tener métodos implementados	Solo declaraciones de métodos
Puede tener propiedades	No puede tener propiedades
Solo se puede heredar una	Se pueden implementar múltiples
Usa extends	Usa implements

Ejemplo Comparativo

```
<?php
// Clase abstracta
abstract class Animal {
    protected $nombre;

    public function __construct($nombre) {
        $this->nombre = $nombre;
    }

    // Método concreto
    public function comer() {
        return "{$this->nombre} está comiendo";
    }

    // Método abstracto
    abstract public function hacerSonido();
}

// Interface
interface Volador {
    public function volar();
}

interface Nadador {
    public function nadar();
}

// Clase que hereda y implementa interfaces
class Pato extends Animal implements Volador, Nadador {
    public function hacerSonido() {
        return "¡Cuac cuac!";
    }

    public function volar() {
```



```

        return "{$this->nombre} está volando";
    }

    public function nadar() {
        return "{$this->nombre} está nadando";
    }
}

$pato = new Pato("Donald");
echo $pato->comer() . "<br>";
echo $pato->hacerSonido() . "<br>";
echo $pato->volar() . "<br>";
echo $pato->nadar();
?>

```

11. Traits

¿Qué son los Traits?

Los traits permiten reutilizar código en múltiples clases sin usar herencia.

```

<?php
trait Logger {
    public function log($mensaje) {
        echo "[" . date('Y-m-d H:i:s') . "] " . $mensaje . "<br>";
    }
}

trait Validador {
    public function validarEmail($email) {
        return filter_var($email, FILTER_VALIDATE_EMAIL) !== false;
    }

    public function validarEdad($edad) {
        return is_numeric($edad) && $edad >= 0 && $edad <= 150;
    }
}

class Usuario {
    use Logger, Validador; // Usar múltiples traits

    private $nombre;
    private $email;

    public function registrar($nombre, $email) {
        if (!$this->validarEmail($email)) {
            $this->log("Error: Email inválido - $email");
            return false;
        }

        $this->nombre = $nombre;
    }
}

```

```

        $this->email = $email;
        $this->log("Usuario registrado: $nombre");
        return true;
    }
}

$usuario = new Usuario();
$usuario->registrar("Juan", "juan@example.com");
?>

```

Resolver Conflictos en Traits

```

<?php
trait A {
    public function metodo() {
        echo "Método de trait A";
    }
}

trait B {
    public function metodo() {
        echo "Método de trait B";
    }
}

class MiClase {
    use A, B {
        B::metodo insteadof A; // Usar el método de B
        A::metodo as metodoA; // Crear alias para el método de A
    }
}

$obj = new MiClase();
$obj->metodo(); // "Método de trait B"
$obj->metodoA(); // "Método de trait A"
?>

```

12. Métodos Mágicos

Métodos Mágicos Comunes

```

<?php
class MiClase {
    private $datos = [];

    // Constructor
    public function __construct() {
        echo "Objeto creado<br>";
    }
}

```

```

// Destructor
public function __destruct() {
    echo "Objeto destruido<br>";
}

// Obtener propiedad inexistente
public function __get($nombre) {
    echo "Intentando obtener: $nombre<br>";
    return $this->datos[$nombre] ?? null;
}

// Establecer propiedad inexistente
public function __set($nombre, $valor) {
    echo "Estableciendo $nombre = $valor<br>";
    $this->datos[$nombre] = $valor;
}

// Verificar si existe propiedad
public function __isset($nombre) {
    return isset($this->datos[$nombre]);
}

// Eliminar propiedad
public function __unset($nombre) {
    unset($this->datos[$nombre]);
}

// Llamar a método inexistente
public function __call($nombre, $argumentos) {
    echo "Llamando a método inexistente: $nombre<br>";
    echo "Argumentos: " . implode(', ', $argumentos) . "<br>";
}

// Convertir objeto a string
public function __toString() {
    return "Instancia de MiClase";
}

// Clonar objeto
public function __clone() {
    echo "Objeto clonado<br>";
}
}

$obj = new MiClase();
$obj->propiedad = "valor"; // Llama a __set()
echo $obj->propiedad;      // Llama a __get()
$obj->metodoInexistente("arg1", "arg2"); // Llama a __call()
echo $obj;                // Llama a __toString()

```

```
$clon = clone $obj;           // Llama a __clone()
?>
```

Ejemplo Práctico con `__get` y `__set`

```
<?php
class Usuario {
    private $datos = [];
    private $camposPermitidos = ['nombre', 'email', 'edad'];

    public function __set($nombre, $valor) {
        if (!in_array($nombre, $this->camposPermitidos)) {
            throw new Exception("Campo no permitido: $nombre");
        }

        // Validaciones específicas
        if ($nombre === 'email' && !filter_var($valor, FILTER_VALIDATE_EMAIL)) {
            throw new Exception("Email inválido");
        }

        if ($nombre === 'edad' && ($valor < 0 || $valor > 150)) {
            throw new Exception("Edad inválida");
        }

        $this->datos[$nombre] = $valor;
    }

    public function __get($nombre) {
        if (!array_key_exists($nombre, $this->datos)) {
            throw new Exception("Campo no existe: $nombre");
        }

        return $this->datos[$nombre];
    }
}

$usuario = new Usuario();
$usuario->nombre = "Juan";
$usuario->email = "juan@example.com";
$usuario->edad = 25;

echo $usuario->nombre; // Juan
?>
```

13. Namespaces

¿Qué son los Namespaces?

Los namespaces organizan el código y evitan conflictos de nombres entre clases.

```

<?php
// Archivo: Modelos/Usuario.php
namespace Modelos;

class Usuario {
    public function obtenerNombre() {
        return "Usuario del modelo";
    }
}
?>

```

```

<?php
// Archivo: Controladores/Usuario.php
namespace Controladores;

class Usuario {
    public function obtenerNombre() {
        return "Usuario del controlador";
    }
}
?>

```

```

<?php
// Usar las clases
require 'Modelos/Usuario.php';
require 'Controladores/Usuario.php';

// Forma 1: Especificar el namespace completo
$modelo = new \Modelos\Usuario();
$controlador = new \Controladores\Usuario();

// Forma 2: Usar "use"
use Modelos\Usuario as ModeloUsuario;
use Controladores\Usuario as ControladorUsuario;

$modelo = new ModeloUsuario();
$controlador = new ControladorUsuario();

echo $modelo->obtenerNombre() . "<br>";
echo $controlador->obtenerNombre();
?>

```

Sub-Namespace

```

<?php
namespace App\Modelos\Usuario;

class Perfil {

```

```
        // ...
    }

    // Usar la clase
    use App\Modelos\Usuario\Perfil;
    $perfil = new Perfil();
    ?>
```

14. Autoloading

Autoload Manual

```
<?php
spl_autoload_register(function($clase) {
    // Convertir namespace a ruta de archivo
    $archivo = str_replace('\\', '/', $clase) . '.php';

    if (file_exists($archivo)) {
        require $archivo;
    }
});

// Ahora se cargan automáticamente
$usuario = new App\Modelos\Usuario();
$producto = new App\Modelos\Producto();
```